

ASD Millennio — Modello logico del database

Database **millennio** (PostgreSQL 16) · MVP R1 (M01 Soci, M02 Calendario/Campi, M04-NFT Palasirio)

Estratto dallo schema reale il 03/07/2026 · Legenda: PK chiave primaria · FK-> chiave esterna · UK unique · IX indice · NN not null

Domini enumerati (ENUM)

```
stato_tessera      = {bozza, in_attesa_pagamento, attiva, scaduta, sospesa}
tipo_consenso      = {privacy, trattamento_dati, foto_video, marketing}
fascia_oraria      = {notte, mattina, pomeriggio}
fascia_oraria_opt  = {notte, mattina, pomeriggio}      (variante nullable = pricing globale)
stato_slot         = {libero, parziale, esaurito, bloccato}
stato_prenotazione = {bloccata, confermata, cancellata, scaduta}
stato_acquisto     = {in_attesa_pagamento, pagato, mintato, fallito, rimborsato}
tipo_pricing_rule  = {tariffa_base, leva_data, leva_scarsita, sconto_promo}
esito_accesso      = {valido, non_valido, slot_errato, tessera_scaduta}
```

M01 — Tesseramento

soci

id	uuid	PK		
nome	varchar	NN		
cognome	varchar	NN		
data_nascita	date	NN		
codice_fiscale	varchar(16)	NN	UK	IX
email	varchar	NN	UK	IX
indirizzo	text			
telefono	varchar			
foto_url	text			
is_minor	boolean	NN		
tutore_id	uuid	FK-> soci.id		(self-ref: tutore -> minori)
sport	varchar[]			(array)
keycloak_user_id	varchar	UK		(ponte con l'auth Keycloak)
created_at	timestampz	NN	default now()	
updated_at	timestampz	NN	default now()	(trigger update_updated_at)

tessere

id	uuid	PK		
socio_id	uuid	NN	FK-> socio.id	IX
numero_tessera	varchar	NN	UK	
sport	varchar	NN		
stato	stato_tessera	NN		
data_emissione	date			
data_scadenza	date			
anno_sportivo	varchar			
pdf_url	text			
created_at	timestampz	NN	default now()	
updated_at	timestampz	NN	default now()	(trigger)

consensi (nessun created_at/updated_at: usa timestamp_firma)

id	uuid	PK		
socio_id	uuid	NN	FK-> socio.id	IX
tipo	tipo_consenso	NN		
testo_versione	varchar			
firmato_da	uuid	FK-> socio.id		(il tutore, per i minori)
timestamp_firma	timestampz			
revocato_at	timestampz			(null = consenso attivo)

M02 — Calendario & Campi

slot_calendario (1 riga per (data, fascia); ore singole in array)

id	uuid	PK	default gen_random_uuid()	
data	date	NN	IX	
fascia	fascia_oraria	NN		
ora_inizio	time	NN		
ora_fine	time	NN		
ore_totali	smallint	NN		
ore_vendute	integer[]	NN	default '{}'	(ore gia' vendute)
ore_in_lock	integer[]	NN	default '{}'	(ore in lock pre-pagamento)
stato	stato_slot	NN	default 'libero'	
nft_token_id	integer			
bloccato_fino_a	timestamptz			
created_at	timestamptz	NN	default now()	
updated_at	timestamptz	NN	default now()	(trigger)
UNIQUE (data, fascia) -> uq_slot_data_fascia				

pricing_rules (standalone · M02, usata da M04 per il prezzo degli slot NFT)

id	uuid	PK	default gen_random_uuid()	
tipo	tipo_pricing_rule	NN		
fascia	fascia_oraria_opt			(null = tutte le fasce)
soglia_min	numeric(8,2)			
soglia_max	numeric(8,2)			
valore	numeric(10,4)	NN		(moltiplicatore o percentuale)
attivo	boolean	NN	default true	
nome	varchar(200)	NN		
valido_fino_a	varchar(10)			(ISO date, null = nessuna scadenza)
note	text			
created_at	timestamptz	NN	default now()	
updated_at	timestamptz	NN	default now()	(trigger)

slot_template_campo (regola ricorrente; disponibilita' on-demand)

id	uuid	PK		
giorno_settimana	smallint	NN		(0=Lun ... 6=Dom)
ora_inizio	time	NN		
ora_fine	time	NN		
num_campi	smallint	NN	default 4	
costo_ora	numeric(8,2)	NN		
sport	varchar[]	NN	default '{}'	
attivo	boolean	NN	default true	
valido_dal	date	NN		
valido_fino_al	date	NN		
note	text			
created_at	timestamptz	NN	default now()	
updated_at	timestamptz	NN	default now()	

prenotazioni_campo

id	uuid	PK		
socio_id	uuid	NN	FK-> soci.id	IX
template_id	uuid	NN	FK-> slot_template_campo.id	IX
data	date	NN	IX	
ora_inizio	time	NN		
ora_fine	time	NN		
campo	smallint	NN		(n. campo 1..num_campi)
importo_eur	numeric(8,2)	NN		
stato	stato_prenotazione	NN	default 'bloccata'	
stripe_session_id	varchar	UK		
bloccata_fino_a	timestamptz			(scadenza lock)
cancellabile_fino_a	timestamptz			
note	text			
created_at	timestamptz	NN	default now()	
updated_at	timestamptz	NN	default now()	

M04 — NFT Palasirio

acquisti_nft (un acquisto = un NFT su piu' ore/slot)

id	uuid	PK		
socio_id	uuid	NN	FK-> soci.id	IX
stripe_session_id	varchar	NN	UK	
stripe_payment_id	varchar		IX (parziale: WHERE not null)	
token_id	integer			(null fino al mint)
contract_address	varchar			
ipfs_uri	varchar			
ical_sha256	varchar			
importo_eur	numeric(10,2)	NN		
stato	stato_acquisto	NN		
acquisto_per_minore	boolean			
minore_id	uuid		FK-> soci.id	
wallet_address	varchar			
metadati_json	text			(backup off-chain IPFS)
ical_content	text			(backup off-chain .ics)
created_at	timestampz	NN	default now()	
updated_at	timestampz	NN	default now()	(trigger)
UNIQUE (contract_address, token_id) -> token unico per contratto				

acquisto_nft_slots (tabella ponte M:N con payload)

acquisto_nft_id	uuid	PK	FK-> acquisti_nft.id	
slot_calendario_id	uuid	PK	FK-> slot_calendario.id	
ore_acquistate	integer[]	NN	default '{}'	(quali ore di quello slot)
PRIMARY KEY (acquisto_nft_id, slot_calendario_id)				(PK composita)

wallet_custodiali (1:1 con socio)

id	uuid	PK		
socio_id	uuid	NN	FK-> soci.id	UK/IX (unique -> 1:1)
wallet_address	varchar	NN	UK	
encrypted_private_key	varchar	NN		(AES-256)
created_at	timestampz	NN	default now()	

accesso_log (audit ISO 27001 verifica QR)

id	uuid	PK		
token_id	integer	NN	IX	(legame logico -> acquisti_nft.token_id)
socio_id	uuid		FK-> soci.id	
slot_key	varchar	NN		
verificato_da	uuid		FK-> soci.id	(lo staff)
esito	esito_accesso	NN		
checked_at	timestampz	NN	default now()	
note	text			

Trasversale

webhook_log (idempotenza Stripe, standalone)

id	uuid	PK		
webhook_id	varchar	NN	UK/IX	(evita doppio processing -> no doppio mint)
event_type	varchar	NN		
payload	text			
processed	boolean			
processed_at	timestampz			
created_at	timestampz	NN	default now()	

Relazioni e cardinalita'

soci	1	---<	N	tessere	(socio_id)
soci	1	---<	N	consensi	(socio_id ; firmato_da -> soci)
soci	1	---<	N	acquisti_nft	(socio_id ; minore_id -> soci)
soci	1	---<	N	prenotazioni_campo	(socio_id)
soci	1	---	1	wallet_custodiali	(socio_id UNIQUE)
soci	1	---	<	soci	(tutore_id, auto-riferimento)
soci	1	---	<	accesso_log	(socio_id, verificato_da)
slot_template_campo	1	---	<	N	prenotazioni_campo (template_id)
acquisti_nft	N	>---	<	N	slot_calendario VIA acquisto_nft_slots (+ ore_acquistate[])
pricing_rules					-> nessuna FK (guida il pricing di slot_calendario, legame logico)

accesso_log.token_id -> acquisti_nft.token_id (legame logico, no FK)
webhook_log -> nessuna FK

Note di design

- **updated_at** gestito da trigger `update_updated_at()` su `soci`, `tessere`, `slot_calendario`, `pricing_rules`, `acquisti_nft`; su `prenotazioni_campo` e `slot_template_campo` è a livello applicativo (SQLAlchemy `onupdate`).
- Granularità oraria **denormalizzata** in array `integer[]` (`ore_vendute`, `ore_in_lock`, `ore_acquistate`) invece di una riga per ora: meno righe e check anti double-sell atomico.
- **token_id** è nullable fino al mint ed è unico *per contratto* (`contract_address`, `token_id`), non globale: un redeploy che riparte da token 0 non collide con lo storico.
- Chiavi primarie **UUID** su tutte le entità; stati come **enum nativi** PostgreSQL.
- Due calendari distinti: **slot_calendario** (diritti d'uso NFT, Lun-Ven 00:00-14:59) e **slot_template_campo/prenotazioni_campo** (prenotazione campi badminton/pickleball).
- **pricing_rules** è al confine **M02-M04**: definita e gestita dal motore di pricing (M02, `modules/calendario/pricing.py`) e dalla dashboard dirigenza; prezza gli **slot NFT** che **M04** vende (l'importo finale finisce in `acquisti_nft.importo_eur`). Non ha FK e **non** riguarda le prenotazioni campi, che usano la tariffa fissa `slot_template_campo.costo_ora`.